Object Oriented Programming ECS414U Project Report

Student: Andrei DODU
Student ID: 250658790

I. *Report Organization*

1. *Introduction*
2. *Typist Class*
   a. Implementing Required Private Fields
   b. Implementing public Methods
   c. Encapsulation
      i. Explanation
      ii. Slideback Validation
      iii. SetAccuracy Validation
   d. Testing Expected Behavior
      i. Test 1
      ii. Test 2
      iii. Test 3
      iv. Test 4
      v. Test 5
3. *TypingRace Class*
   a. Explaining Behavior
   b. Bug Identification and Fixing
      i. Bug 1
      ii. Bug 2
      iii. Bug 3
      iv. Bug 4
      v. Bug 5
      vi. Bug 6
      vii. Bug 7
      viii. Bug 8

```
  TYPING RACE — passage length: 40 chars
==============================================
|                                          ①| TURBOFINGERS (Accuracy: 0.78)
|                                  ②        | QWERTY_QUEEN (Accuracy: 0.7)
|                              ③            | HUNT_N_PECK (Accuracy: 0.7)
==============================================
  [zz] = burnt out    [<] = just mistyped

And the winner is... TURBOFINGERS!
Final accuracy: 0.8 improved from 0.78
```

II. Report Main

1. Introduction

This Report covers Part I of the Typing Race Simulator Project for ECS414U. The aim of Part I was to complete and improve upon the inherited code from our colleague Ty Posaurus. Our colleague gave us two incomplete or faulty classes, incomplete Typist.java and faulty TypingRace.java. Typist was a barebone not even functional record representing every individual competitor whilst TypingRace which was representing the race itself was mostly functional except for some glaring game breaking bugs. Our goal during this Part I of the project was to improve upon the inherited code and make it function properly and stylishly while keeping a git repository to prove our gradual work on it.

2. Typist Class

The Typist Class represents one competitor in the typing race. Each typist has their own name, display symbol, progress value and burnout state. The assignment requires us to include 6 fields and several public methods controlling race behavior.

a. Implementing Required Fields

Our first task was to implement these following fields in the Typist class:

```
private String name;  // Name of the Typist
private char symbol; // Symbol that represents typist in the race
private int progress; // Progress in the typist
private boolean burnOut; // Y/N if typist is burnt out
private int burnoutTurnsRemaining; // For how long the typist is gonna be burnt out/ remaining
private double accuracy; // accuracy of typist
```

These variables were all declared private to ensure the safety of the internal state of each Typist, as to not allow modification except through the clear methods available to us. It is basic OOP procedure. Following the implementation of these fields we used a constructor to assign them base values:

```
public Typist(char typistSymbol, String typistName, double typistAccuracy) {
    name = typistName;
    progress = 0;
    burnOut = false;
    burnoutTurnsRemaining = 0;
    symbol = typistSymbol;

    setAccuracy(typistAccuracy);
}
```

This constructor initializes our Typist with the values given to it and base values like progress = 0 or burnOutTurnsRemaining = 0; It is a vital part in the construction of our Typist Class.

b. Implementing Public Methods

The Typist class contains several public methods, which in most cases are setters and getters, will allow other classes, such as TypingRace, to interact with a Typist while still maintaining security with no interaction through the private fields.

Our main accessors	Our main setters
getAccuracy() getProgress() getName() getSymbol() getBurnoutTurnsRemaining() isBurntOut()	setAccuracy() setSymbol() typeCharacter() slideBack() burnOut() recoverFromBurnout() resetToStart()

These methods allow other classes to access the values of our private fields such as name through getName or symbol through getSymbol. These methods will allow our other classes to modify the data of our typists securely. Per example we have resetToStart(): which resets all our Typists progress, burnout or burnout duration values to the beginning, making a new race possible.

c. Encapsulation

i. Explanation

Now onto encapsulation is used in the Typist class by making all fields private. This means that external classes cannot directly intervene or change important values such as progress accuracy or burnout state, instead our class provides these public methods that can control how values are accessed or changed. This protects our class from Human Error or invalid data. For example, another developer cannot by mistake change a value like accuracy into an invalid data type (like -10).

ii. Slideback validation

The slideBack() method is particularly important for encapsulation because typists can move backwards when they mistype however their progress should never be negative. The slideBack() method is :

```
public void slideBack(int amount)
{
    if (amount > 0) {
        if (progress - amount < 0 ) {
            progress = 0;
        }else {
            progress = progress - amount;
        }
    }
}
```

The method first checks if the slide amount is positive then it checks whether subtracting the amount would make the progress fall bellow 0, if so it would just set it to 0, otherwise it continues as normal with progress – amount.

This validation is necessary because negative progress would not make sense in a typing race, it also protects our race display from showing invalid positions. All in all this validation is a clear example of why encapsulation is necessary, if we allowed progress to be modified anywhere without worry, this validation could be forgotten resulting in unwanted results.

iii. setAccuracy validation

Now similar to our last method, setAccuracy() also includes a validation. This validation functions like a clamp making sure the values for accuracy aren't under 0 or over 1.00.

```
public void setAccuracy(double newAccuracy)
  {
    if(newAccuracy < 0.0) {
      newAccuracy = 0.0;
    }else if (newAccuracy > 1.0) {
      newAccuracy = 1.0;
    }

    accuracy = Math.round(newAccuracy * 100.0) / 100.0;
  }
```

Now, we do this validation because our accuracy is used in probability calculations, as such invalid values would make the simulation unfair, impossible or logically incorrect (you cannot have negative accuracy). The rounding at the end ensures that the value won't have long decimal outputs caused by java double precision.

d. Testing expected Behavior

i. Test 1 Progress

This test checks if the typeCharacter() does work.

Tested Code:

```
        Typist t = new Typist('①', "TEST", 0.80);

    System.out.println(t.getProgress());
    t.typeCharacter();
    System.out.println(t.getProgress());
```

Expected Result : 0 1

```
(base) andreidodu@Bogdans-MacBook-Air-M1 TypingRaceSimulator %  cd /Users/andreidodu/OOPGit/TypingRaceSimulator ; /usr/bin/env /Library/Java/Jav
aVirtualMachines/temurin-25.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/andreidodu/Library/Application\ Suppor
t/Code/User/workspaceStorage/27933f1703ba3943f9d9a23250d90f31/redhat.java/jdt_ws/TypingRaceSimulator_3743cf6d/bin testingcode
0
1
```

Verified

ii. Test 2 slideBack Boundary

This test checks if the slideBack boundary is functional

Tested code:

```
        Typist t = new Typist('①', "TEST", 0.80);
```

```
t.slideBack(5);
System.out.println(t.getProgress());
```

Expected Result: 0

```
(base) andreidodu@Bogdans-MacBook-Air-M1 TypingRaceSimulator %  cd /Users/andreidodu/OOPGit/TypingRaceSimulator ; /usr/bin/env /Library/Java/Jav
aVirtualMachines/temurin-25.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/andreidodu/Library/Application\ Suppor
t/Code/User/workspaceStorage/27933f1703ba3943f9d9a23250d90f31/redhat.java/jdt_ws/TypingRaceSimulator_3743cf6d/bin testingcode
0
(base) andreidodu@Bogdans-MacBook-Air-M1 TypingRaceSimulator %
```

Verified

iii. Test 3 Burnout countdown

This test checks if the burnout countdown is functional

Tested Code:

```
Typist t = new Typist('①', "TEST", 0.80);

    t.burnOut(3);

    System.out.println(t.isBurntOut() + " " + t.getBurnoutTurnsRemaining());

    t.recoverFromBurnout();
    System.out.println(t.isBurntOut() + " " + t.getBurnoutTurnsRemaining());

    t.recoverFromBurnout();
    System.out.println(t.isBurntOut() + " " + t.getBurnoutTurnsRemaining());

    t.recoverFromBurnout();
    System.out.println(t.isBurntOut() + " " + t.getBurnoutTurnsRemaining());
```

Expected Result: true 3 / true 2 / true 1 / false 0

```
(base) andreidodu@Bogdans-MacBook-Air-M1 TypingRaceSimulator %  cd /Users/andreidodu/OOPGit/TypingRaceSimulator ; /usr/bin/env /Library/Java/Jav
aVirtualMachines/temurin-25.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/andreidodu/Library/Application\ Suppor
t/Code/User/workspaceStorage/27933f1703ba3943f9d9a23250d90f31/redhat.java/jdt_ws/TypingRaceSimulator_3743cf6d/bin testingcode
true 3
true 2
true 1
true 0
```

Verified

iv. Test 4 resetToStart

This test checks if resetToStart is functional

Tested Code:

```
Typist t = new Typist('①', "TEST", 0.80);

    t.typeCharacter();
    t.typeCharacter();
    t.burnOut(3);

    System.out.println(t.getProgress());
    System.out.println(t.isBurntOut());
    System.out.println(t.getBurnoutTurnsRemaining());

    t.resetToStart();

    System.out.println(t.getProgress());
    System.out.println(t.isBurntOut());
```

```
System.out.println(t.getBurnoutTurnsRemaining());
```

Expected result: 2 true 3 0 false 0

```
(base) andreidodu@Bogdans-MacBook-Air-M1 TypingRaceSimulator %  cd /Users/andreidodu/OOPGit/TypingRaceSimulator ; /usr/bin/env /Library/Java/Jav
aVirtualMachines/temurin-25.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/andreidodu/Library/Application\ Suppor
t/Code/User/workspaceStorage/27933f1703ba3943f9d9a23250d90f31/redhat.java/jdt_ws/TypingRaceSimulator_3743cf6d/bin testingcode
2
true
3
0
false
0
```

v. Test 5 Accuracy Clamping

This test checks if accuracy clamping is functional

Tested Code:

```
Typist t = new Typist('①', "TEST", 0.80);

    t.setAccuracy(1.5);
    System.out.println(t.getAccuracy());

    t.setAccuracy(-0.5);
    System.out.println(t.getAccuracy());
```

Expected Result : 1.0 / 0.0

```
0
(base) andreidodu@Bogdans-MacBook-Air-M1 TypingRaceSimulator %  cd /Users/andreidodu/OOPGit/TypingRaceSimulator ; /usr/bin/env /Library/Java/Jav
aVirtualMachines/temurin-25.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMessages -cp /Users/andreidodu/Library/Application\ Suppor
t/Code/User/workspaceStorage/27933f1703ba3943f9d9a23250d90f31/redhat.java/jdt_ws/TypingRaceSimulator_3743cf6d/bin testingcode
1.0
0.0
(base) andreidodu@Bogdans-MacBook-Air-M1 TypingRaceSimulator %
```

3. *TypingRace Class*

TypingRace controls the race itself, it stores universal values like the passage length and the three typists themselves, then runs the simulation turn by turn

a. Explaining Behavior

The TypingRace constructor receives the race length when created

```
public TypingRace(int passageLength)
  {
    this.passageLength = passageLength;
        seat1Typist = null;
        seat2Typist = null;
        seat3Typist = null;
    }
```

As such a race can only have one set length and when it's finished a new race must be started. To add typist we must use one of the public methods of the class called addTypist() which looks like this

```
public void addTypist(Typist theTypist, int seatNumber)
    {
        if (seatNumber == 1)
        {
            seat1Typist = theTypist;
        }
        else if (seatNumber == 2)
        {
```

```
            seat2Typist = theTypist;
        }
        else if (seatNumber == 3)
        {
            seat3Typist = theTypist;
        }
        else
        {
            System.out.println("Cannot seat typist at seat " + seatNumber + " —
there is no such seat.");
        }
    }
```

As you can see this method is hard coded and not versatile, allowing for a maximum of 3 racers any attempt to add more will be rejected through our checks. After creating our 3 typists we must use the public method race.startRace(), with a small check to ensure that all 3 racers exist the race officially starts

After the race ends all 3 racers are reset to the beginning to allow a new race to start.

```
seat1Typist.resetToStart();
seat2Typist.resetToStart();
seat3Typist.resetToStart();
```

All racers advance through the private method advanceTypist(), which can allow them to recover from a burnout, type a character, mistype or burn out. All of these are based on probability from the racer's stats. The race will end when one racer satisfies the winning condition >= passageLength.

At the end the winner's accuracy is improved and he is announced as the winner.

b. Bug Identification and Fixing

i. Bug 1 Seat 3 Reset

The Original startRace() method reset only seat1Typist and seat2Typist, it did not reset seat 3 typist.

Fix:

```
seat1Typist.resetToStart();
seat2Typist.resetToStart();
seat3Typist.resetToStart();
```

ii. Bug 2 Race Crash Seat Empty

The Original Program assumed that all three seats had typists. If any seat was empty the program would crash by calling methods on null. As such a simple fix was implemented :

Fix:

```
if (seat1Typist == null || seat2Typist == null || seat3Typist == null) {
            System.out.println("Cannot start race: all three seats must have
typists.");
            return;
```

```
        }
```

As such without all 3 seats being present the program will not be allowed to start.

iii. Bug 3 Backwards mistype Probability

The Original mistype calculation was
*Math.random() < theTypist.getAccuracy() * MISTYPE_BASE_CHANCE*

But this made high accuracy typists more likely to mistype which does not make sense in the context of the exercise

As such I changed to formula to :

```
Math.random() < (1.0 - theTypist.getAccuracy()) * MISTYPE_BASE_CHANCE
```

Which mathematically fixes the formula

iv. Bug 4 Incorrect Finish Condition

The Original finish condition was :
if (theTypist.getProgress() == passageLength)

But this is too strict, if progress ever reached or passed the passage length then he would win, in the assignment it is specifically said that the typist should win if he reaches or exceeds the passage length. As such I changed this victory condition to

```
theTypist.getProgress() >= passageLength
```

v. Bug 5 Winner was not announced

The original code had a TODO comment showing where the winner must be announced and printed out but it did not actually print it out. It seems that Ty forgot to do it.

As a fix I added a winner variable;
Typist winner = null;
Then after the race I did
System.out.println("And the winner is... " + winner.getName() + "!");

vi. Bug 6 Burnout Accuracy Missing

Our specification states that after a burnout the typist's accuracy should slowly decrease. The original code did cause burnouts but did not reduce the typist accuracy.

I added this line inside advanceTypist:

```
theTypist.setAccuracy(theTypist.getAccuracy() - 0.02);
```

vii. Bug 7 Winner accuracy improvement

The Specification states that a winner should have their accuracy improved. The original code did not update that winner's accuracy.

The fix was simple :

```
winner.setAccuracy(OldAccuracy+0.02); // As Shown in the image of the simulation,
0.02 improvement.
```

viii. Bug 8 Incorrect Burnout Display

The original output legend shows that the Burnout Marker should look like this [zz] but in the code we display burnout as the typist's symbol followed by ~

As such we have to fix the display

```
if (theTypist.isBurntOut())
        {
            System.out.print("zz");
            spacesAfter-= 2 ; // symbol + zz together take two characters
        }
```

Verifying :

```
 ========================================
 |                                ⓪     | TURBOFINGERS (Accuracy: 0.78)
 |                  ②zz                | QWERTY_QUEEN (Accuracy: 0.68) BURNT OUT (0 turns)
 |                        ③             | HUNT_N_PECK (Accuracy: 0.7)
 ========================================
   [zz] = burnt out    [<] = just mistyped
```

III. Conclusion

In conclusion, Part I of the Typing Race Simulator was completed by implementing the missing Typist class and improving the TypingRace class so the simulation works correctly. The final version uses encapsulation to protect typist data, validates progress and accuracy, handles burnout, resets typists properly, detects the winner using the correct >= condition, and applies accuracy changes after winning or burning out. The bugs found in the original code were identified, explained, and fixed where possible, resulting in a more reliable and fair command-line typing race simulation.